Mentimeter

Laborator 10

Prolog: Introducere

Mentimeter

Prolog

limbaj logic, descriptiv, care permite specificarea problemei de rezolvat în termenii unor fapte cunoscute despre obiectele universului problemei și ai relațiilor existente între aceste obiecte

Mentimeter

Entitatile limbajului

- fapte
- reguli
- scopuri (intrebari)

Mentimeter

Ipoteza lumii inchise

Tot ceea ce nu este cunoscut sau nu poate fi demonstrat este considerat a fi fals

Mentimeter

Fapte

- predicate de ordinul 1 (cu oricate argumente) considerate adevarate (**true => propozitie**)
- ex: nada()., node(a)., areLab('semigrupaX', 'oraY', 'materiaZ').

Mentimeter

Reguli

- forma: propozitie :- dep1, dep2, ..., depn.
- cum se traduce asta? (dep1 si dep2 si ... si depn) => propozitie
- ex: trece(X, Nota) :- Nota >= 5.

Mentimeter

```
student(x).
student(y).
student(z).

%% alte varianta care **pica pe unele situatii**(depinde de ce vreau):
%%      trece(N) :- N >= 5.
%%
%% trece(valoare_constanta) se calculeaza corect, dar daca incerc sa aflu toate valorile N pentru care trece(N).
%% atunci o sa am urmatoarea eroare: "Arguments are not sufficiently instantiated".
%% (intuitiv: >= verifica valoarea din stanga cu cea din dreapta si aici nu am marginit deloc N-ul)
trece(N) :- member(N, [5, 6, 7, 8, 9, 10]).
```

Exemplu



Mentimeter

Scopuri

- intrebari la care trebuie sa raspundem cu da(true) sau nu(false)
- predicate pentru care dorim sa stim valoarea de adevar **in functie de cunostintele pe care le avem**
- **cu variabile legate sau nu**

Mentimeter

```
%% concat are 3 parametri(concat/3) si e de forma concat(elem, list, newList).
%% [X | L] = lista care are head-ul X si tail-ul L (X primul parametru e acelasi cu X din [X | L])
concat(X, L, [X | L]).

%% in terminal:
%% 1. concat(1, [2], [1, 2]). -> intoarce true si numai verifica daca predicatul este satisfiabil
%%                               pentru parametri dati (1, [2], [1, 2])
%% 2. concat(1, [2], X). -> am fixat elem si list si pe baza regulii concat o sa caute toate valorile
%%                          pe care le poate lua newList pentru ca concat(1, [2], newList) sa fie true
%%                       -> intoarce X = [1, 2].
%% 3. concat(1, X, Y). -> am fixat numai elem si o sa cautam valorile pentru X(list) si Y(newList)
%%                        pentru care concat(1, X, Y). este true
%%                     -> intoarce Y = [1 | X] (X poate fi orice lista)
```

Exemplu

Mentimeter

Structuri

- aceeasi structura ca la fapte: nume_structura(nume_camp1, nume_camp2, ...)
- daar diferenta e ca structurile apar ca argumente ale predicatelor

Mentimeter

```
%% la urmatoarea apare "Warning: Singleton variables: [Nume]" pentru ca Nume apare o singura data in regula
%% (numai il declar, nu il folosesc => pot sa il sterg).
%% trece(student(Nume, Nota)) :- member(Nota, [5, 6, 7, 8, 9, 10]).
trece(student(Parcurs, Examen)) :- Parcurs >= 3, Examen >= 2.

%% in terminal cand apelez trebuie sa folosesc si numele structurii
%% trece(student(3, 2)).
```

Exemplu



Mentimeter

Liste

- empty: []
- cu un numar fix de elemente: [X], [X, Y], [X1, X2, ..., Xn]
- [X | L] lista cu head-ul X si tail-ul L (lista trebuie sa contina cel putin un element - X)
- [X1, X2, ..., Xn | L] lista cu X1, ..., Xn concatenate la L (lista are minim cele n valori Xi)

Mentimeter

Operatori

- aritmetici: + - * /
- logici: ,(and) ;(or) \+(negatie)
- relationali: =\= < > =< >= =:= == \== is

Mentimeter

= vs == vs =:= vs is

- = unificare intre cele 2 expresii (rezulta o lista de legari)
- == verifica daca cele 2 expresii au aceeasi structura (**fara sa le evalueze**)
- =:= evalueaza expresiile din stanga/dreapta si verifica daca rezultatele sunt egale
- is atribuie valoarea expresiei din dreapta (**numai expresia din dreapta se evalueaza**)

Mentimeter

Unificare X = Y

- X, Y pot fi orice (atomi, variabile, structuri)
- atomi = constante, simboluri, [] (ceva ce nu poate fi divizat mai mult)
- negatie: X\= Y (X nu unifica cu Y)

Mentimeter

Reguli unificare

- X, Y atomi: X = Y daca sunt X, Y sunt la fel (ex: 1 = 1, dar 1 \= 2)
- X variabila, Y orice: X = Y unifica prin legarea lui X la Y (X = [1, 2] => X este legat la [1, 2])
- in rest: incercam sa gasim o instantiere a variabilelor astfel incat sa obtinem 2 termeni egali din punct de vedere structural

Mentimeter

```
%% 1 = 1.          => true
%% 1 = 1 + 0.      => false (chiar daca au aceeasi valoare, nu sunt identici structural)
%% X = 1.          => X = 1. (X este legat la 1)
%% X = 1 + 0.      => X = 1 + 0. (**nu se evalueaza** 1 + 0)
%% X + 0 = 1 + 0. => X = 1.
%% X + 1 = 1 + 0. => false (daca X = 0 as obtine aceeasi valoare in stanga/dreapta
%%                    numai daca le evaluez, dar aici nu se face evaluare si se cauta
%%                    exact aceeasi structura iar 0 + 1 \= 1 + 0)
%% X + 1 = 1 + Y. => X = Y, Y = 1.
```

Exemple =



Mentimeter

```
%% facem o functie consecutive care verifica daca 2 elemente sunt consecutive (diferenta = 1)

consecutive1(X, X + 1).
%% consecutive1(1, 2). intoarce false, dar consecutive1(1, 1 + 1). intoarce true
%% X + 1 = se cauta valorile care au forma asta, fara sa se faca evaluarea expresiei X + 1

consecutive2(X, Y) :- Y = X + 1.
%% consecutive2(1, 2). intoarce false (in spate se incearca 2 = 1 + 1, iar 2 si 1 + 1 nu unifica)
%% consecutive2(1, 1 + 1). intoarce true pentru ca Y(1 + 1) unifica cu X + 1(1 + 1)

consecutive3(X, Y) :- Y is X + 1.
%% consecutive3(1, 2). intoarce true pentru ca se evalueaza X + 1(1 + 1) la 2 care este egal cu Y(2).
%% consecutive3(1, 1 + 1). intoarce false pentru ca **Y nu se evalueaza**(ramane 1 + 1), X + 1 da
%%                         (se obtine 2) si se verifica daca 1 + 1 este egal cu 2 => false

consecutive4(X, Y) :- Y == X + 1.
%% consecutive4(1, 2). intoarce false (verificare structurala, iar 2 \== 1 + 1)
%% consecutive4(1, 1 + 1). intoarce true (prin inlocuire obtinem 1 + 1 == 1 + 1)

consecutive5(X, Y) :- Y =:= X + 1.
%% consecutive5(1, 2). intoarce true pentru ca =:= forteaza evaluarea lui Y(care ramane 2) si
%%                     X + 1(devine 2) si valorile lor sunt egale
%% consecutive5(1, 1 + 1). intoarce true din acelasi motiv: se evalueaza si Y(1 + 1 => 2)
%%                         si X + 1(1 + 1 => 2) si obtinem aceeasi valoare
```

Exemplu pentru toti operatorii de egalitate ish

Mentimeter

```
day(Name) :- Name = 'Monday'; Name = 'Tuesday'; Name = 'Wednesday'.
notDay(Name) :- \+day(Name).

of(_) :- false.
notOf(Ceva) :- \+of(Ceva).
```

;(sau), \+ (negatie)

Mentimeter

Super mare grija la \+

- \+ prop(ceva) intoarce true doar daca **nu exista niciun ceva a.i. prop(ceva)**
- 1. notDay: avem 3 stringuri (Monday, Tuesday, Wednesday) pe care day(Name). => exista cel putin o valoare X pe care day(X). => \+day(X) este false
- 2. notOf: of intoarce mereu false => nu exista nicio valoare X pentru care of sa fie satisfiabil => \+of(X) este true

Mentimeter

Negari

- daca vreau sa verific ca A != B pot sa o scriu ca A \= B (cele 2 nu unifica)
- sau folosind dif(A, B)
- atat \= cat si dif verifica daca sunt egale structural (ex: dif(1, 1 + 0) intoarce false)

Mentimeter

Conventii

- variabilele incep cu litera mare
- constantele(simbolurile) incep cu litera mica
- predicat/nrArgumente (predicatul are nrArgumente argumente)
- predicat(+Arg1, -Arg2, ?Arg3, ..., +ArgN) (+ = input, - = output, ? = input/output)

Mentimeter

```
greaterThan1(Value, List) :- Value > Elem, member(Elem, List).
greaterThan2(Value, List) :- member(Elem, List), Value > Elem.
```

Super mare grija la ordine

Mentimeter

Ce face functia?

- verifica daca **exista cel putin un element din lista mai mare decat valoarea data**
- varianta 1 genereaza o eroare pentru ca **scopurile** se incearca satisfacute pe rand
- daca incepem cu Value > Elem atunci incercam sa le evaluam inainte sa stim cine este Elem (ERROR: Arguments are not sufficiently instantiated)

Mentimeter

Ordinea conteaza

Se incearca satisfacerea scopurilor pe rand
(se pun intr-o stiva si se foloseste
backtracking pentru a le parcurge)
=> pot diferi foarte mult rezultatele numai de
la schimbarea ordinii

Mentimeter

```
deadline(pp).
deadline(pc).

are(ana, pp).
mergeBal1(X) :- deadline(Y), \+are(X, Y).
mergeBal2(X) :- \+are(X, Y), deadline(Y).
```

Exemplu demo

?